SESION-006

Respaldos y Recuperación ante Ataques

Introducción

Este informe explica para qué sirven los códigos de la guía del Módulo 6, que trata sobre auditoría de cambios, copias de seguridad y recuperación de una base de datos. Todos los ejemplos se hacen sobre `juego_online` y mezclan sentencias SQL con scripts de Bash que se ejecutan desde la terminal.

Objetivo

La idea es explicar qué hace cada bloque de código y sus instrucciones principales, dejando claro cuáles cosas se hacen desde MySQL/phpMyAdmin y cuáles desde la terminal del sistema operativo.

Diferencia entre SQL y Bash

No es lo mismo, y conviene tenerlo claro desde el principio. SQL lo usamos para crear tablas, consultar o modificar registros y definir triggers. Bash, en cambio, sirve para automatizar tareas del sistema: generar archivos de respaldo, organizar carpetas y llamar a las herramientas de MySQL. Por eso, los archivos `.sh` de esta guía no se pegan en la pestaña SQL de phpMyAdmin, sino que se ejecutan en la terminal.

Código 1: Auditoría mediante TRIGGER

Archivo: `scripts/01_auditoria_trazas.sql`. Su función es dejar registro automático cada vez que se tocan las monedas de un jugador: guarda el valor anterior, el nuevo, la cuenta de base de datos que hizo el cambio y la fecha del evento.

- `USE juego_online;` — selecciona la base sobre la que vamos a trabajar.

- `CREATE TABLE IF NOT EXISTS Auditoria_Monedas (...);` — crea la tabla de auditoría, pero solo si todavía no existe.

- `auditoria_id INT AUTO_INCREMENT PRIMARY KEY` — le da un identificador único y automático a cada registro.

- `jugador_id INT NOT NULL` — indica a qué jugador afecta el cambio y obliga a que ese campo no quede vacío.

- `monedas_antes` / `monedas_despues` — guardan cuántas monedas había antes y cuántas quedaron después.

- `operacion VARCHAR(20)` — dice qué tipo de operación fue; en este ejemplo siempre será `UPDATE`.

- `usuario_bd VARCHAR(100)` — conserva la cuenta de MySQL/MariaDB que ejecutó el cambio.

- `fecha DATETIME DEFAULT CURRENT_TIMESTAMP` — anota sola la fecha y hora del evento, sin que tengamos que hacer nada.

- `DROP TRIGGER IF EXISTS auditoria_monedas;` — borra el trigger anterior si ya existía, para poder crearlo de nuevo sin errores.

- `DELIMITER //` — cambia el delimitador de forma temporal, porque el cuerpo del trigger tiene sus propios `;` internos y sin esto se cortaría antes de tiempo.

- `CREATE TRIGGER ... AFTER UPDATE ON Jugadores` — define la acción automática que se ejecuta después de actualizar filas de `Jugadores`.

- `FOR EACH ROW` — hace que el trigger se dispare por cada fila que se vea afectada.

- `OLD.monedas` y `NEW.monedas` — `OLD` es el valor de antes y `NEW` el de después de la actualización.

- `IF NOT (OLD.monedas <=> NEW.monedas)` — comprueba si el valor realmente cambió. El operador `<=>` compara de forma segura incluso cuando hay `NULL` de por medio.

- `INSERT INTO Auditoria_Monedas ...` — mete en la auditoría el valor viejo, el nuevo, la operación y el usuario.

- `CURRENT_USER()` — saca la cuenta de base de datos que está ejecutando la operación.

- `DELIMITER ;` — devuelve el delimitador normal para el resto de sentencias.

- `UPDATE Jugadores SET monedas = monedas + 100 WHERE jugador_id = 1;` — le suma 100 monedas al jugador 1; sirve como prueba para ver si el trigger responde.

- `SELECT * FROM Auditoria_Monedas ORDER BY fecha DESC;` — muestra los registros de auditoría, del más reciente al más viejo.

Resultado esperado: si las monedas cambian, el trigger agrega un registro. Si el valor queda igual, no se añade nada nuevo a la auditoría.

Código 2: Respaldo completo con mysqldump

Archivo:`scripts/02_backup_completo.sh`. Lo que hace es generar una copia lógica de `juego_online` en un archivo `.sql` que después sirve para reconstruir la base si hace falta.

- `#!/bin/bash` — le dice al sistema que el archivo debe correr con Bash.

- `set -euo pipefail` — corta el script si hay errores, si se usa una variable sin definir o si falla algo dentro de una tubería.

- `DB_NAME="juego_online"` — indica qué base se va a respaldar.

- `DB_USER="root"` — indica con qué usuario se conecta.

- `BACKUP_DIR="$(dirname "$0")/../respaldos"` — calcula la carpeta de respaldos a partir de donde está el script.

- `FECHA="$(date +%Y%m%d_%H%M%S)"` — toma la fecha y hora actuales para que cada respaldo tenga un nombre distinto.

- `ARCHIVO=...` — arma el nombre del `.sql` con la fecha y la hora dentro.

- `mkdir -p "$BACKUP_DIR"` — crea la carpeta de destino si no existe.

- `echo "Generando respaldo..."` — muestra un mensaje en pantalla para saber que el proceso arrancó.

- `mysqldump -u "$DB_USER" -p --single-transaction --routines --triggers "$DB_NAME" > "$ARCHIVO"` — exporta la base al archivo indicado.

  - `-u` es el usuario.

  - `-p` pide la contraseña de forma interactiva.

  - `--single-transaction` consigue una copia consistente sin bloquear las tablas InnoDB durante todo el proceso.

  - `--routines` incluye las rutinas almacenadas.

  - `--triggers` incluye los triggers.

  - `>` manda la salida al archivo.

- `echo "Respaldo completo generado..."` — avisa al final en qué ubicación quedó el archivo.

Resultado esperado se genera un `.sql` dentro de `respaldos`. La contraseña se pide en el momento y nunca hay que escribirla dentro del script.

Código 3: Binary log y recuperación punto en el tiempo (PITR)

Archivo: `scripts/03_backup_incremental_binlog.sh`. Este archivo documenta cómo se configura el binary log y cómo se recupera la base hasta un momento anterior a un incidente. El binlog guarda los cambios confirmados y, sumado a un respaldo completo, permite recuperar lo que pasó después del respaldo.

- `BINLOG_ARCHIVE_DIR=...` — define la carpeta donde se van a archivar los binary logs.

- `mkdir -p "$BINLOG_ARCHIVE_DIR"` — crea esa carpeta si no existe.

- `cat <<'EOF' ... EOF` — imprime un bloque de configuración en la terminal. Ojo: no aplica esa configuración al servidor por sí solo.

- `[mysqld]` — es la sección de configuración del servidor MySQL.

- `log_bin = /var/log/mysql/mysql-bin.log` — activa el binary log y dice dónde guardarlo (siempre que se coloque en el archivo real de configuración).

- `server_id = 1` — le pone un identificador al servidor.

- `binlog_format = ROW` — configura el registro de cambios por filas.

- `expire_logs_days = 7` — en esta configuración de referencia, los binlogs expiran a los siete días; si la versión del servidor lo permite, claro.

- `mysqlbinlog --stop-datetime="..." archivo-binlog | mysql ... juego_online` — lee los eventos hasta la fecha y hora indicadas y los manda al cliente `mysql` para que los aplique sobre la base restaurada.

Importante: este script es, sobre todo, una guía. Crear una carpeta y mostrar el bloque con heredoc no activa el binary log por arte de magia. Para eso hay que tocar `my.cnf` o `my.ini` con permisos de administrador y conservar los binlogs.

Por cierto, PITR significa *Point-In-Time Recovery*, o recuperación a un punto en el tiempo. Se restaura primero el respaldo completo y después se reproducen los eventos del binlog hasta el momento que nos interesa, normalmente justo antes del incidente.

Respaldos programados, respaldos de objetos y snapshots

Respaldo programado con cron

- `crontab -e` — abre la configuración de tareas programadas del usuario (en los sistemas que lo soportan).

- `0 1 * * * comando` — programa la tarea para que corra todos los días a la 1:00 a. m. Los campos son: minuto, hora, día del mes, mes y día de la semana.

- `>> /var/log/backup_juego.log 2>&1` — agrega la salida al log y también redirige los mensajes de error.

- `crontab -l` — muestra las tareas que están programadas.

La idea con esto es automatizar las copias y no depender de que alguien se acuerde de correrlas.

Respaldo de objetos individuales

- `mysqldump -u root -p juego_online Auditoria_Monedas > respaldo_auditoria.sql` — exporta solo la tabla `Auditoria_Monedas`.

- `mysqldump -u root -p --no-data --routines --triggers juego_online > respaldo_objetos.sql` — exporta solo la estructura y los objetos que se pidan, sin llevarse los datos de las tablas.

- `mysql -u root -p juego_online < respaldo_auditoria.sql` — importa las sentencias del archivo a `juego_online`; el `<` redirige el contenido del archivo hacia `mysql`.

Snapshot o imagen de disco

Un snapshot es una copia del estado de un disco o volumen que administra la infraestructura. A diferencia del `.sql` que genera `mysqldump`, esta es una copia a nivel de almacenamiento. Sirve para recuperar rápido un volumen o un servidor, pero no reemplaza los respaldos lógicos verificables.

Código 4: Simulacro de desastre y recuperación

Archivo: `scripts/04_simulacro_desastre_recuperacion.sh`. Lo que hace es comprobar si la base se puede reconstruir desde el respaldo más reciente. Guarda un estado de referencia, elimina la base de prueba, la vuelve a crear, importa el respaldo y compara los resultados.

- `DB_NAME="juego_online"` y `DB_USER="root"` — definen la base y el usuario del ejercicio.

- `ULTIMO_RESPALDO="$(ls -t ... | head -n1)"` — busca los respaldos que coinciden y se queda con el más reciente según la fecha de modificación.

- `if [[ -z "$ULTIMO_RESPALDO" ]]; then ... exit 1` — si no encuentra ningún respaldo, detiene el proceso para no borrar la base sin tener una copia a mano.

- `SELECT nombre_usuario, nivel, monedas FROM Jugadores ORDER BY jugador_id;` — consulta los datos de los jugadores que se van a usar como referencia antes de la prueba.

- `tee estado_antes.txt` — muestra el resultado en pantalla y guarda una copia en un archivo de texto.

- `DROP DATABASE juego_online;` — borra la base completa. Esto es destructivo y solo debe hacerse en un entorno de pruebas con respaldo verificado.

- `CREATE DATABASE ... CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;` — vuelve a crear la base vacía con la configuración de caracteres indicada.

- `mysql ... < "$ULTIMO_RESPALDO"` — importa el archivo SQL para reconstruir tablas, datos y objetos que vienen en el respaldo.

- `tee estado_despues.txt` — guarda el estado consultado después de la restauración.

- `diff -q estado_antes.txt estado_despues.txt` — compara los dos archivos. Si son idénticos, no hay diferencias en los datos que se consultaron.

Si prefieres hacerlo desde lo gráfico, la alternativa es Exportar, Eliminar e Importar en phpMyAdmin. Eso sí, la comparación solo verifica las columnas que se consultaron (`nombre_usuario`, `nivel` y `monedas`); no prueba por sí sola que todos los objetos de la base estén íntegros.

JavaScript de la página interactiva

El HTML del módulo también trae JavaScript para controlar la guía desde el navegador. Ese código cambia los pasos visibles, muestra las explicaciones, copia los fragmentos y corre las simulaciones locales.

- `document.addEventListener("DOMContentLoaded", ...)` — espera a que el documento esté cargado antes de buscar elementos y asociar eventos.
- `querySelectorAll(".step")` y `querySelectorAll(".panel")` — seleccionan los botones de navegación y los paneles de contenido.
- `activate(targetId)` — activa el paso que se eligió, oculta los demás paneles y actualiza el fragmento de la dirección del navegador.
- `addEventListener("click", ...)` — conecta funciones a los clics de los botones, como abrir ventanas o cambiar de sección.
- `explainData` — guarda las explicaciones línea por línea de los scripts SQL y Bash.
- `navigator.clipboard.writeText(text)` — intenta copiar el código al portapapeles; si falla, hay un plan B.
- `triggerForm` y `MONEDAS_ACTUALES` — simulan el resultado de un cambio de monedas y muestran si se registraría una traza; no tocan la base real.
- `renderBackupFilename()` — arma un nombre de ejemplo con la fecha y hora del navegador; no genera ningún `.sql`.
- `renderPitr()` — actualiza una línea de tiempo simulada para mostrar qué transacciones se recuperarían.
- `runDrill()` — representa en el navegador los pasos del simulacro con datos de ejemplo; no ejecuta `DROP DATABASE` ni se conecta a MySQL.
- `resetBtn.addEventListener("click", ...)` — limpia los resultados visuales para poder volver a correr la simulación.

Hay que tener clara la diferencia: los bloques de SQL/Bash describen cosas que se ejecutarían en el servidor; los simuladores de JavaScript solo cambian lo que ves en el navegador.

Conclusiones

- El trigger registra solo los cambios reales en las monedas y deja una traza de auditoría confiable.

- `mysqldump` sirve para respaldos completos o de objetos puntuales, y con ellos se restauran datos y estructuras.

- El binary log, junto con un respaldo completo, permite recuperar hasta un punto anterior al incidente.

- `cron` se encarga de los respaldos periódicos, mientras que los snapshots dan recuperación a nivel de disco.

- El simulacro comprueba el proceso de restauración y compara una selección de datos antes y después.

- Borrar una base es destructivo: las pruebas deben hacerse en un entorno local o de laboratorio, y solo después de verificar que el respaldo sirve.

Respaldo completo de `juego_online`

Objetivo

Hacer un respaldo completo de la base `juego_online` que incluya estructura, datos, procedimientos y disparadores, sin interrumpir el servicio.

Herramientas

- `mysqldump`

- phpMyAdmin

- Motor InnoDB

Método 1 — Automatizado (código)

Archivo `02_backup_completo.sh`

Comando:

```bash
mysqldump -u root -p --single-transaction --routines --triggers juego_online > ../respaldos/juego_online_completo_FECHA.sql
```

- `--single-transaction` — respalda sin bloquear tablas, así los jugadores siguen jugando mientras se hace la copia.
- `--routines` — incluye los procedimientos y funciones.
- `--triggers` — incluye los disparadores.

Método 2 — Gráfico (phpMyAdmin)

1. Entrar a phpMyAdmin → base `juego_online` → **Exportar**
2. Método: **Personalizado** → Formato: **SQL**
3. Seleccionar todo
4. Activar **Agregar sentencia `CREATE PROCEDURE` / `FUNCTION` / `EVENT`**
5. Activar **Agregar sentencia `CREATE TRIGGER`**
6. Clic en **Exportar**

El archivo que se descarga, `juego_online.sql`, es equivalente al que genera el script.

Archivo generado

`/respaldos/juego_online_completo_20241027.sql`

Verificación: dentro del archivo se ven los `CREATE TABLE`, los `INSERT INTO` y los `CREATE DEFINER='root'@'localhost' PROCEDURE` / `TRIGGER`, que es como phpMyAdmin exporta las rutinas.

Prueba de restauración

Se creó la base `juego_online_prueba`, se le dio a *Importar*, se seleccionó el archivo de respaldo y la importación salió bien. Con eso se confirma que el respaldo funciona.

Conclusión

Quedó un respaldo completo por dos vías: por código, para poder automatizarlo, y por phpMyAdmin, para hacer un respaldo manual rápido cuando se necesite.